Queen Mary
University of London

EBU5376
Digital Signal Processing

Introduction to DSP Systems

Module Organizer: Dr Yixuan Zou yixuan.zou@qmul.ac.uk

Outline

- What are DSP Systems?
- Mathematical Representations of Systems
- Different Types of DSP Systems
  - Linear Time-Invariant (LTI) Systems

Outline

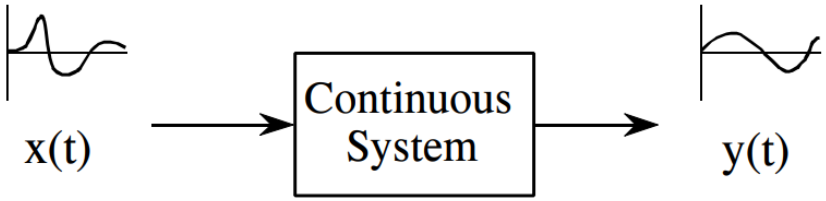
- What are DSP Systems?
- Mathematical Representations of Systems
- Different Types of DSP Systems
  - Linear Time-Invariant (LTI) Systems

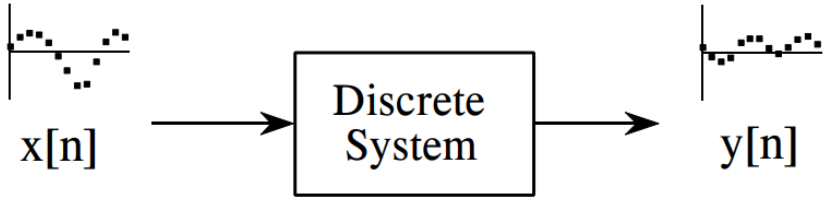
DSP Systems – What are they?

- Definition: In DSP, a system is any process that produces an output signal in response to an input signal.
- DSP systems can be used to perform a wide range of signal processing tasks, such as filtering, modulation, demodulation, compression, equalization...
- A DSP system can be characterized by its input-output relationship, which can be represented by a mathematical model or a block diagram.

DSP Systems

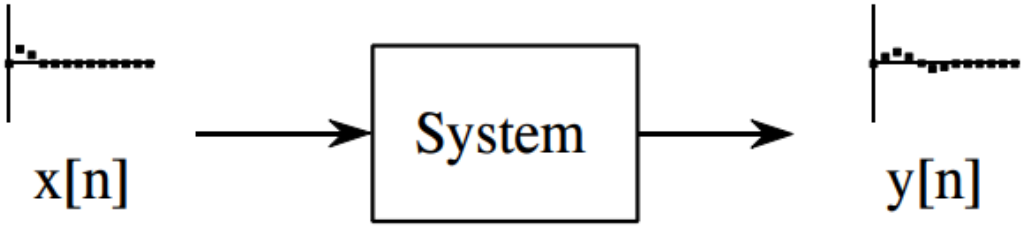
- Continuous systems input and output continuous signals, such as in analog electronics
- Discrete systems input and output discrete signals, such as computer programs that manipulate the values stored in arrays.





DSP Systems – Notations

- In this module, we are concerned with the discrete systems
- The input of a discrete system is often denoted as $x[n]$
- The corresponding output is denoted as $y[n]$



DSP Systems – Why interested?

- Why do we need **mathematical models** for DSP systems?
- How do we **analyze, optimize, and control** DSP systems effectively?
- Understanding DSP Systems allows us to:
  - Establish mathematical models to describe system behavior.
  - Analyze system properties like stability, response, and performance.
  - Optimize and design better DSP solutions for real-world applications.
  - Control system behavior to meet desired performance criteria.

Outline

- What are DSP Systems?
- Mathematical Representations of Systems
- Different Types of DSP Systems
  - Linear Time-Invariant (LTI) Systems

Difference Equation

- Definition: The difference equation expresses the outputs $y[n], y[n-1], \ldots$ of the system as a function of inputs $x[n], x[n-1], \ldots$.

- The standard form of a difference equation for an LTI system can be represented as:

$$y[n] + a_1 y[n-1] + \cdots + a_N y[n-N]$$
$$= b_0 x[n] + b_1 x[n-1] + b_M x[n-M]$$

where $a_1, \ldots, a_N$ and $b_0, \ldots, b_M$ are the coefficients of the difference equation.

- M and N are known as the memory lengths for input $x[n]$ and output $y[n]$.

Difference Equation – Example

Example 1: Modeling an Amplifier in a DSP System

- Many DSP applications require **amplifying or attenuating a signal** by a fixed factor, such as in audio processing or communication systems.
- A simple amplifier can be mathematically represented as:

$$y[n] = 2x[n]$$

where the output is always **twice the amplitude** of the input signal.

Difference Equation – Example

Example 2: Modeling a Delay System in DSP

- Many DSP applications require **delaying a signal** for purposes such as echo effects in audio, time alignment in communication systems, and synchronization in sensor networks.
- A simple delay can be mathematically represented as:

$$y[n] = x[n-k]$$

where the output is a **time-shifted version** of the input by k samples

Difference Equation

- The standard form

$$y[n] + a_1 y[n-1] + \cdots + a_N y[n-N] = b_0 x[n] + b_1 x[n-1] + b_M x[n-M]$$

can be reformulated

$$y[n] = -a_1 y[n-1] - \cdots - a_N y[n-N] + b_0 x[n] + b_1 x[n-1] + \cdots + b_M x[n-M]$$

into the <u>recursive form</u>:

$$y[n] = -\sum_{i=1}^{N} a_i y[n-i] + \sum_{j=0}^{M} b_j x[n-j]$$

- The **standard form** describes the input-output relationship of a system, suitable for theoretical analysis, while the **recursive form** explicitly shows how the current output is computed from past outputs and current inputs, making it ideal for practical implementation.

Difference Equation - Recursive

$$y[n] = -\sum_{i=1}^{N} a_i y[n-i] + \sum_{j=0}^{M} b_j x[n-j]$$

- When the coefficients a_i of the past outputs are non-zero, the difference equation is **recursive**
- It means, to calculate $y[n]$, we need to calculate $y[n-1]$, $y[n-2], \ldots, y[1], y[0]$.
- The system needs to store all past outputs and use them to compute the current output → Time-consuming

Difference Equation - Recursive

- To address this, we can reformulate the recursive difference equation into a non-recursive equation, that produces the same outputs
- This process is known as solving a difference equation
- To solve a recursive equation, we must have a starting point, known as the initial condition.
- Have a try at solving the following difference equation:

$$y[n] = 0.25y[n-1], \qquad y[-1] = 1$$

Difference Equation - Recursive

$$y[n] = 0.25y[n-1], \qquad y[-1] = 1$$

n	$y[n]$
-1	1
0	
1	
2	
3	
…	
n	

Outline

- What are DSP Systems
- Mathematical Representations of Systems
- Different Types of DSP Systems
  - Linear Time-Invariant (LTI) Systems

Types of DSP Systems

A system may exhibit more than one properties at the same time

- Common types of DSP systems:

1. Linear vs. Nonlinear Systems
2. Time-Invariant vs. Time-Variant Systems
3. Causal vs. Non-Causal vs. Anti-Causal Systems
4. Memoryless vs. Systems with Memory
5. Stable vs. Unstable vs. Marginally Stable Systems

Linear Systems

- Most useful systems fall into a category called linear systems
- <u>Definition</u>: A linear DSP system is a system that satisfies the principle of superposition. Mathematically, we have

$$x_1(n) \xrightarrow{\text{results in}} y_1(n)$$

$$x_2(n) \xrightarrow{\text{results in}} y_2(n)$$

$$x_1(n) + x_2(n) \xrightarrow{\text{results in}} y_1(n) + y_2(n)$$

$$c_1 x_1(n) + c_2 x_2(n) \xrightarrow{\text{results in}} c_1 y_1(n) + c_2 y_2(n)$$

Linear Systems - Example

- <u>Example</u>: Given a system with input-output relationship as $y[n] = 3x[n]$, is this a *linear* system?

Linear Systems - Example

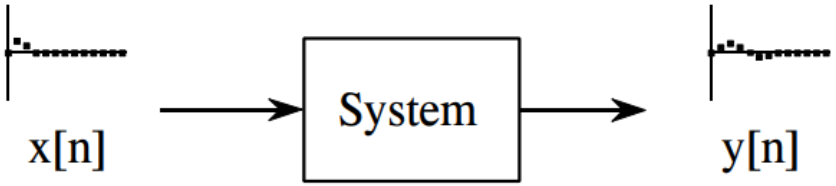
- <u>Example</u>: Given a system with input-output relationship as $y[n] = 3x[n] + 4$, is this a *linear* system?

Linear Systems - Properties

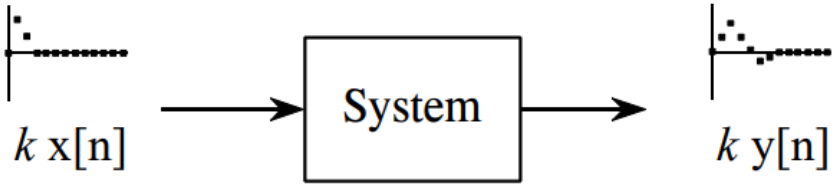
- Linear systems are **homogeneous**

  - an amplitude change in input results in an identical amplitude change in the output.

IF

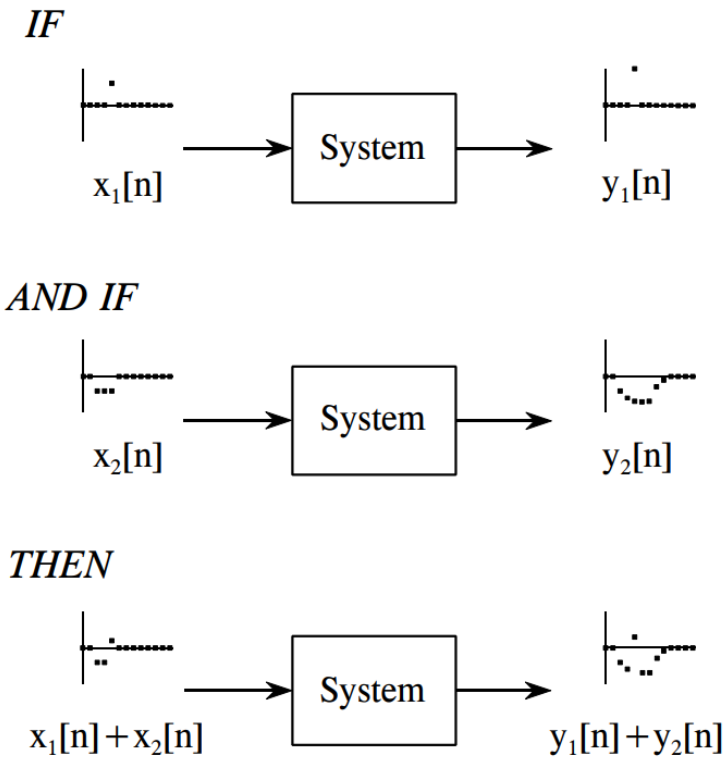


THEN



Linear Systems - Properties

- Linear systems are **additive**
  - composite signals pass through it without interacting



Linear Systems - Properties

- If you can show that a system is both homogeneous and additive, then you have proven that the *system is linear*.

- Likewise, if you can show that a system doesn't have one or both properties, you have proven that *the system is non-linear*.

Linear Systems - Example

- Example: Given a system with input-output relationship as $y[n] = 3x[n] + 4$, is this a *linear* system? Prove it.

Why Linearity Matters?

- Many of the most powerful tools in DSP, such as the Fourier Transform and Z-Transform, rely on linearity to function correctly
- The **Fourier Transform is a linear operator**, meaning that for any two signals $x_1[n]$ and $x_2[n]$, and scalars a and b, we have:

$$\mathcal{F}(ax_1[n] + bx_2[n]) = a\mathcal{F}(x_1[n]) + b\mathcal{F}(x_2[n])$$

Why This Matters:

- This **superposition property** allows us to analyze complex signals as sums of simpler components (sinusoids).
- If a system is **linear**, we can determine its response to **any input** by analyzing how it responds to individual frequency components.
- **Filtering in the frequency domain** (e.g., applying a low-pass filter) relies on the assumption that the system follows linearity.

Non-Linear Systems Also Useful

Dynamic Range Compression in Audio Processing

- **A linear amplifier** simply scales the signal up or down, maintaining the same relative amplitude differences.
- **A nonlinear compressor** selectively reduces the volume of loud sounds while keeping quiet sounds intact
- This is used in **radio broadcasting, hearing aids, and streaming services** like Spotify and YouTube to ensure audio remains at a comfortable level.
- A compressor maps input amplitudes to output amplitudes nonlinearly, often following a function like:

$$y[n] = x[n]^{\gamma}, \qquad 0 < \gamma < 1$$

Where γ controls the degree of compression. Unlike a linear gain system where $y[n] = kx[n]$, this nonlinear mapping **suppresses loud peaks more than quiet parts.**

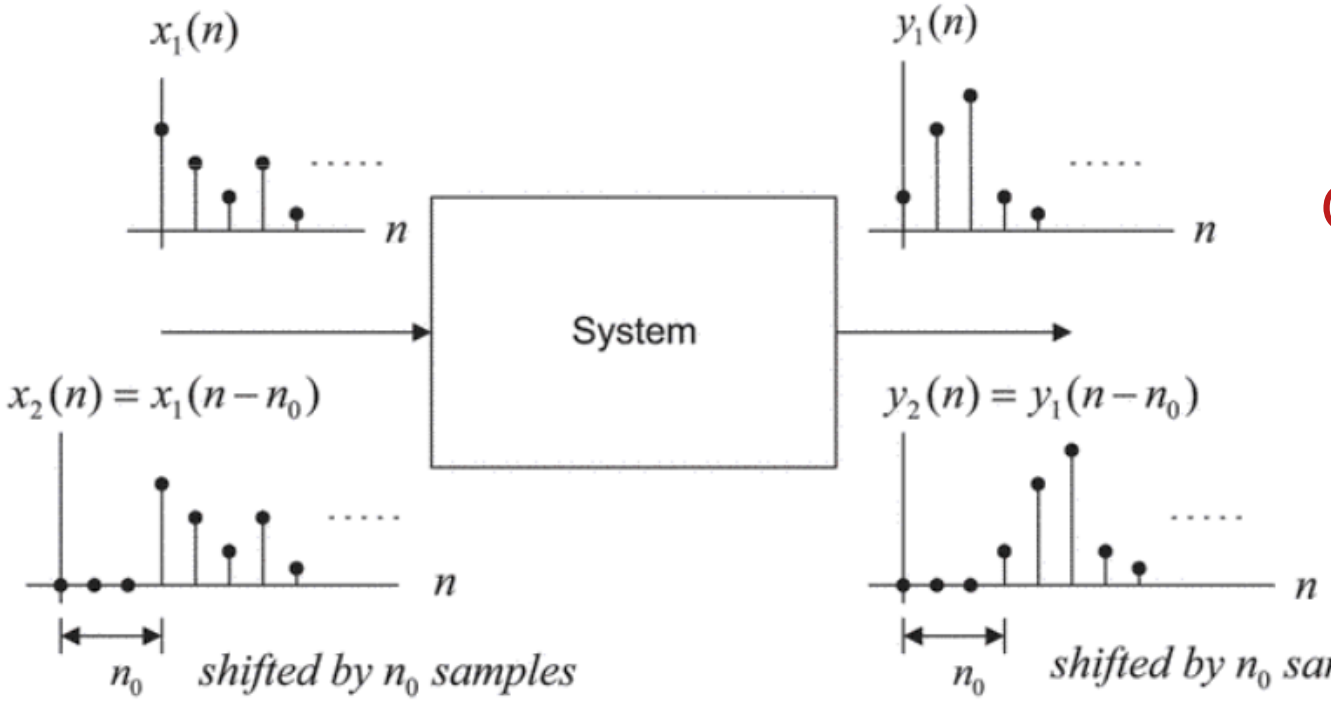
Time-invariant Systems

- Definition: A time-invariant system is one where a time delay (or shift) in the input sequence causes an equivalent time delay in the system's output sequence

$$x(n) \xrightarrow{\text{results in}} y(n)$$

$$x'(n) = x(n+k) \xrightarrow{\text{results in}} y'(n) = y(n+k)$$

Time-invariant Systems



Original Signal

Time-shifted Signal

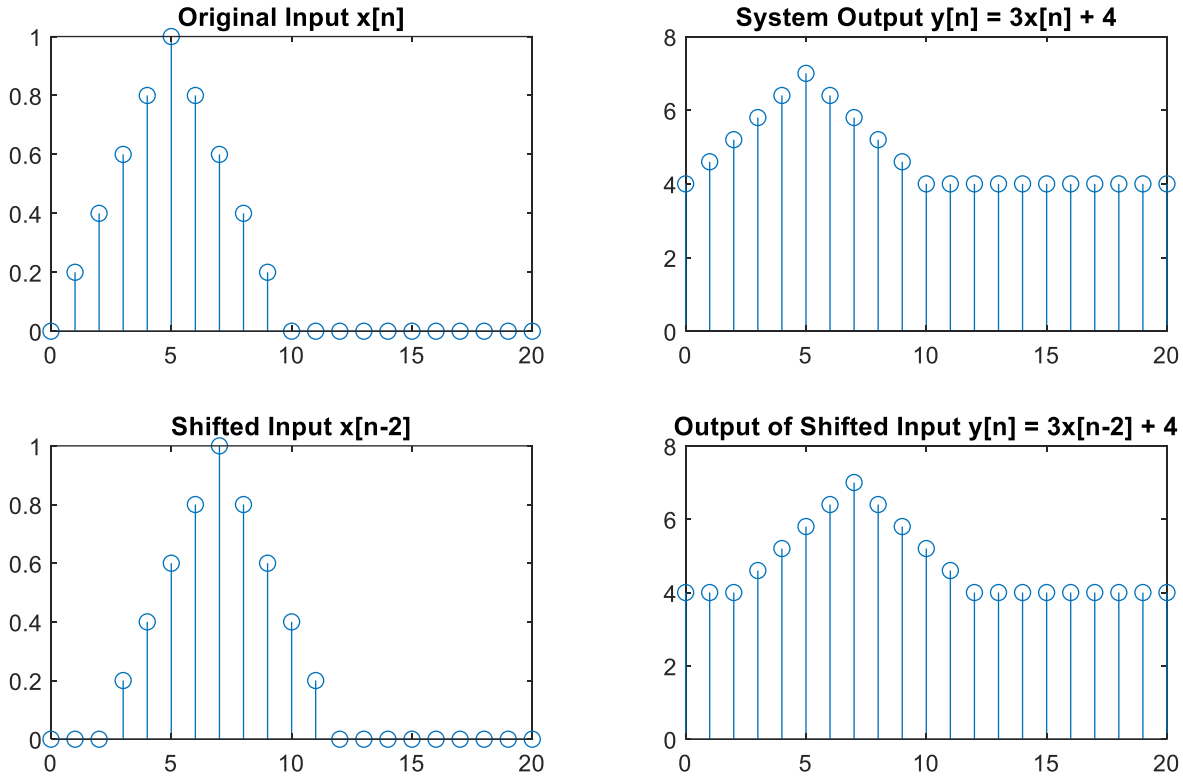
Time-invariant Systems- Example

- Example: Given a system with input-output relationship as $y[n] = 3x[n] + 4$, is this a *time-invariant* system?

Time-invariant Systems- Example

- Example: Given a system with input-output relationship as $y[n] = 3x[n] + 4$, is this a *time-invariant* system?

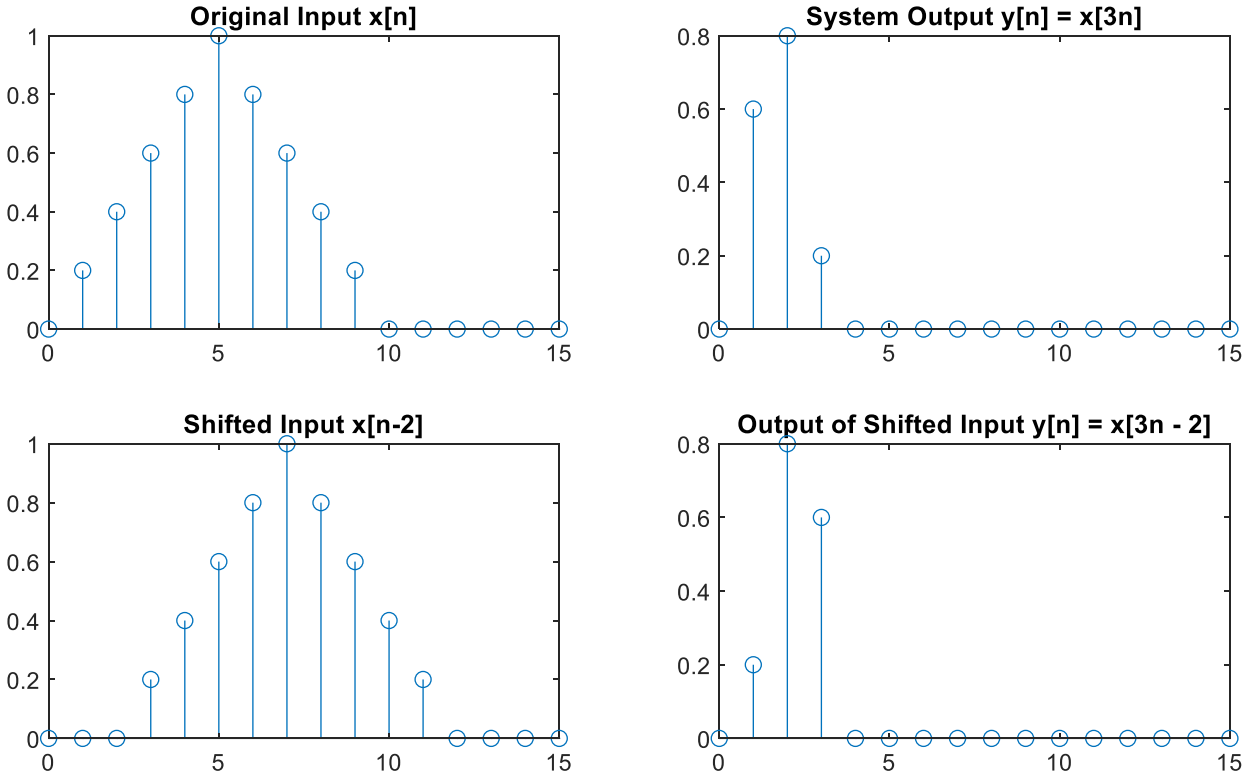
Time-invariant Systems- Example



Time-invariant Systems- Example

- Example: Given a system with input-output relationship as $y[n] = x[3n]$, is this a *time-invariant* system?

Time-invariant Systems- Example

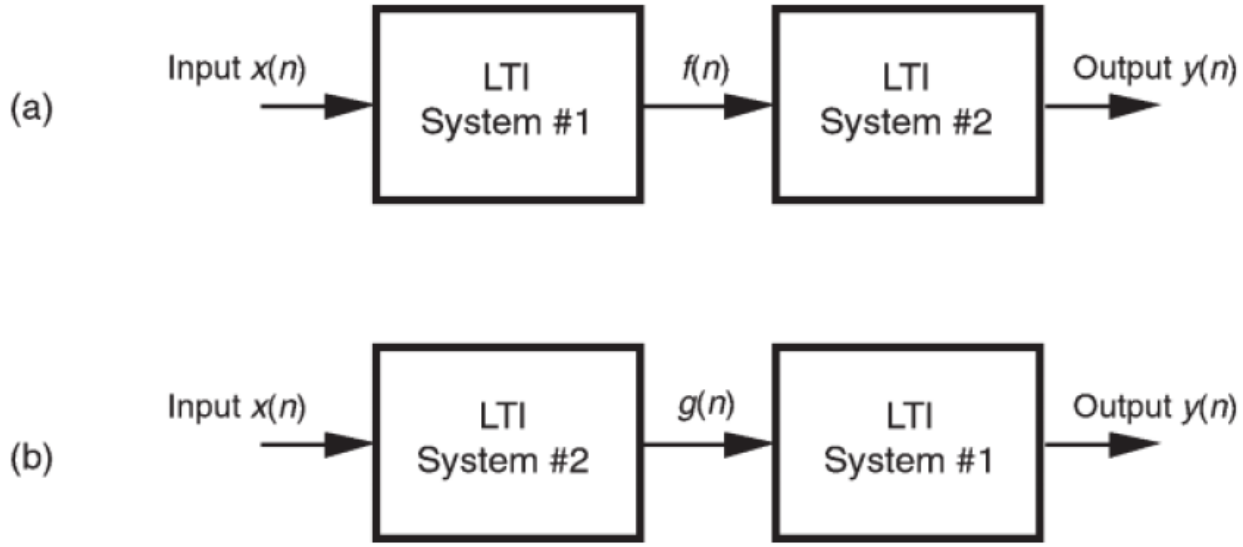


Linear Time-Invariant (LTI) Systems

- If a DSP system is both linear and time-invariant, it is called a linear time-invariant (LTI) system
- LTI system's behavior is constant over time. This means that properties, such as its impulse response and transfer function remain unchanged over time.
  - E.g., The system $y[n] = 3x[n] + 4$ is time-invariant but not linear, hence is not a LTI system
- LTI systems inherit the properties of linear systems, meaning that LTI systems are homogeneous and additive

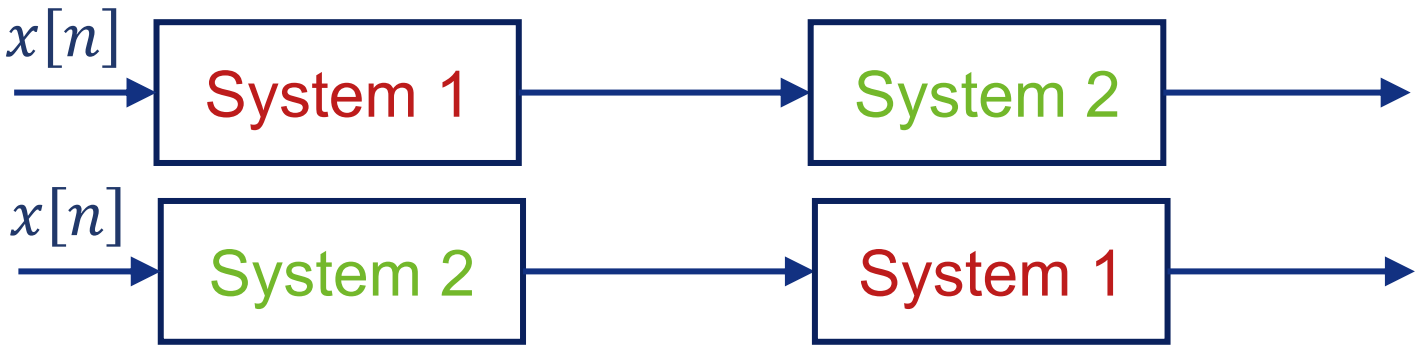
Linear Time-Invariant (LTI) Systems - Properties

- LTI systems are also **commutative**
  - swapping the order of two cascaded systems does not alter the final output



Linear Time-Invariant (LTI) Systems - Properties

- Example: Given two LTI systems:
  - System 1: $y[n] = 3x[n]$; System 2: $y[n] = \frac{1}{2}x[n]$



Linear Time-invariant (LTI) Systems

- In DSP, we focus our study on LTI systems because LTI systems are **mathematically well-defined**
  1. Superposition and convolution apply – The response to a complex input can be broken down into simpler signals and summed using convolution.
  2. Fourier and Z-transform techniques work efficiently – LTI systems can be efficiently studied in both time and frequency domains.
  3. Predictable stability and frequency response – LTI systems have well-defined conditions for stability and can be fully characterized using poles and zeros.

Other Useful Types of Systems

- Causal, Non-causal, Anti-causal systems
- Memoryless systems
- Stable, Unstable, Marginal Stable systems

* LTI systems can have these properties.
* For example, $y[n] = ax[n]$ is a stable, causal, and memoryless LTI system.

Causal Systems

Not to be mixed up with the word "Casual"

- A causal system is a system where the output $y[n]$ **does not depend on the future inputs**.
  - In other words, it only depends on present and/or past inputs $x[n], x[n-1], x[n-2], \ldots$
- A non-causal system is a system where the output $y[n]$ **depends on the future inputs**.
- An anti-causal system is a system where the output $y[n]$ **depends only on the future inputs**.
- Causal systems can be implemented in real-time, as they only require present and/or past inputs to compute the output.
- Most real-world systems are causal. Non-causal and anti-causal systems are mainly *theoretical* or used in offline processing, where future data is available (e.g., post-processing of recorded signals).

Causal Systems

- <u>Example:</u> State whether the following systems are causal/non-causal/anti-causal systems

1. $y[n] = 3x[n] + 4$
2. $y[n] = 2x[n] + 5x[n-2]$
3. $y[n] = 0.5x[n-1] + 0.5x[n+1]$
4. $y[n] = 3x[n+1] + 1$

Memoryless Systems

- Definition: A memoryless system is a system where the output at any given time **depends only on the present inputs.**
  - E.g., A light switch; A simple amplifier; A temperature sensor
  - Otherwise, the system has memory

- To determine this property, think about
  - whether the system needs to memorize/store the inputs?

Memoryless Systems

- <u>Example:</u> Describe the following systems in terms of memory/memoryless.

1. $y[n] = 3x[n] + 4$
2. $y[n] = 2x[n] + 5x[n-2]$
3. $y[n] = 0.5x[n-1] + 0.5x[n+1]$
4. $y[n] = 3x[n+1] + 1$

Stability of DSP Systems

- In Signal and Systems, you probably studied the concept of bounded-input bounded-output (BIBO) stability.
- The definition of stability in this course generalizes BIBO stability
- We define three states of stability
  - Stable — BIBO stable
  - Marginal stable — Not BIBO stable
  - Unstable — Not BIBO stable

- Don't worry if you don't know BIBO stability. We will study our definition from scratch
- In this module, the stability of a system is evaluated based on the poles of the transfer function
- We will come back to this once we understand what is a transfer function

Summary

- <u>Common types of DSP systems:</u>
  1. Linear systems
  2. Time-invariant systems
  3. Causal / Non-causal / Anti-causal systems
  4. Memoryless systems
  5. Stable / Unstable / Marginally Stable systems

Summary

- We are interested in LTI system because it is easy to design, and the output is predictable
- Difference equation is often used to represent the input-output relationship in LTI systems
- However, difference equations are often recursive
- Solving the difference equation meaning writing the output $y[n]$ independent of the past outputs $y[n-1], y[n-2], \ldots$
- The main method to solve a difference equation is z-transform

Queen Mary
University of London